

Chương 2:

MAPREDUCE và HADOOP cơ bản

Nguồn tham khảo:
Slides Khoa học dữ liệu lớn – Phùng Quốc Định
Slides Data-Intensive Text Processing with MapReduce

Nguyễn Ngọc Điệp

Nội dung

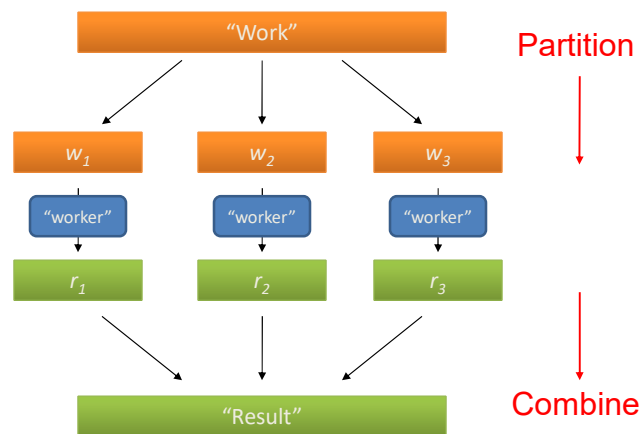
- Giới thiệu
- Nguyên lý lập trình hàm
- Mapper và Reducer
- Phân tách và kết hợp (Partitioners and Combiners)
- Hệ thống tệp phân tán (Distributed File System)
- Kiến trúc phân cụm Hadoop (Hadoop Cluster Architecture)

Nguyễn Ngọc Điệp

Giới thiệu: Sự khác biệt

Nguyễn Ngọc Diệp

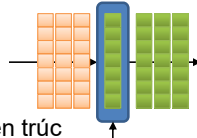
CHIA ĐỂ TRI (Divide and Conquer)



Khá phức tạp...

Các vấn đề căn bản

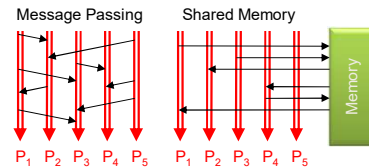
scheduling, data distribution, synchronization, inter-process communication, robustness, fault tolerance, ...



Các vấn đề về kiến trúc

Flynn's taxonomy (SIMD, MIMD, etc.), network typology, bisection bandwidth UMA vs. NUMA, cache coherence

Các mô hình lập trình khác nhau



Các cấu trúc lập trình khác nhau

mutexes, conditional variables, barriers, ...
masters/slaves, producers/consumers, work queues, ...

Các vấn đề chung

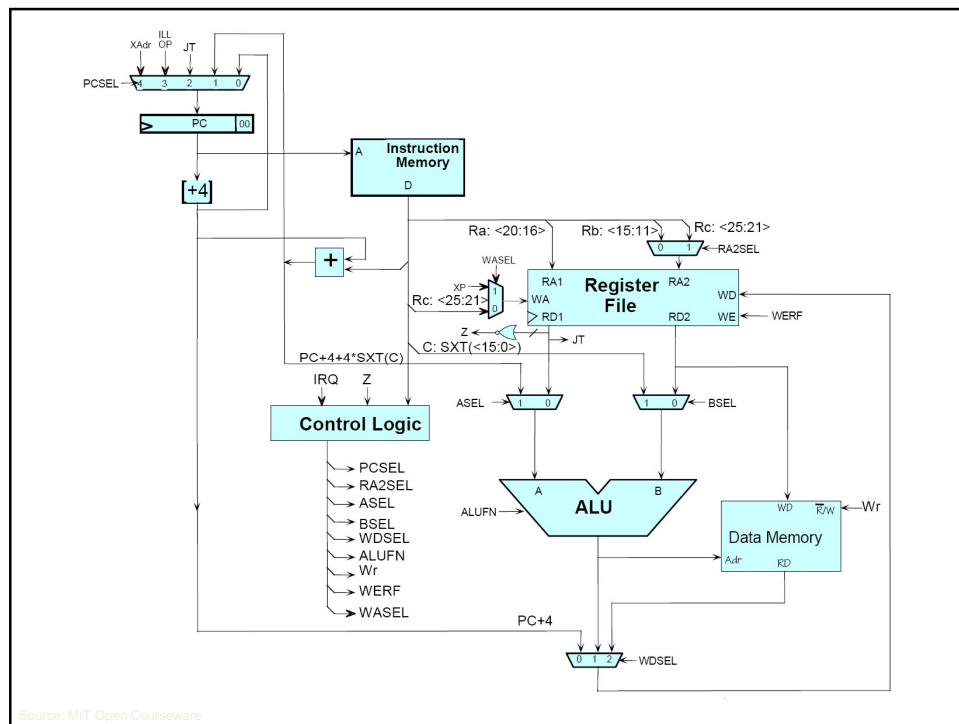
livelock, deadlock, data starvation, priority inversion...
dining philosophers, sleeping barbers, cigarette smokers, ...

Thực tế: lập trình viên gánh vác gánh nặng quản lý về xử lý đồng thời (concurrency)

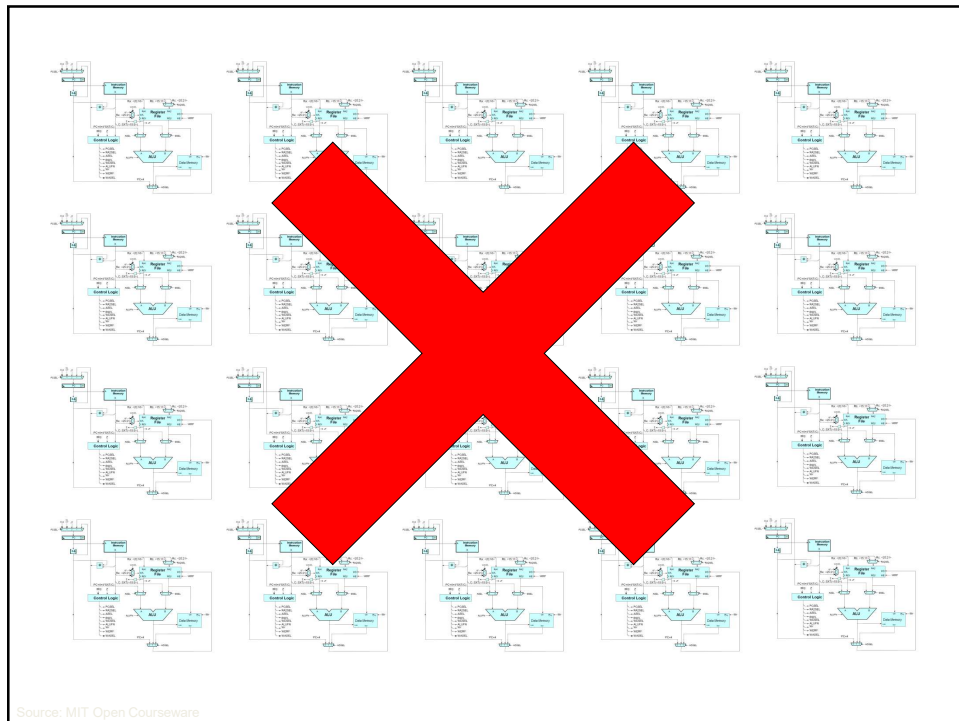
Nguyễn Ngọc Diệp



Source: Ricardo Guimarães Hermann



Source: MIT Open Courseware



Source: MIT Open Courseware

Nguyên lý lập trình hàm

Nguyễn Ngọc Điệp

Khủng hoảng của phần mềm

- Làm thế nào để giải quyết vấn đề về kích cỡ và độ phức tạp của các chương trình máy tính hiện đại?
- Làm thế nào để giảm thời gian và chi phí phát triển chương trình?
- Làm thế nào để đảm bảo các chương trình hoạt động chính xác?

Nguyễn Ngọc Điệp

Các ngôn ngữ lập trình

- Một cách tiếp cận là thiết kế các ngôn ngữ lập trình mới:
 - Cho phép các chương trình được viết rõ ràng, chính xác và ở mức độ trừu tượng cao
 - Hỗ trợ các thành phần phần mềm có thể tái sử dụng
 - Khuyến khích sử dụng kiểm tra theo chuẩn

Nguyễn Ngọc Điệp

- Cho phép tạo mẫu nhanh;
- Cung cấp các công cụ giải quyết vấn đề mạnh mẽ.



Các ngôn ngữ lập trình hàm/chức năng (functional programming languages) cung cấp một framework phù hợp để giải quyết các mục tiêu này.

Lập trình hàm là gì?

- Mô hình lập trình xem việc tính toán là sự đánh giá các hàm toán học và tránh sử dụng trạng thái và các dữ liệu biến đổi.
- Nhấn mạnh việc ứng dụng hàm số
 - Trái với phong cách lập trình mệnh lệnh, nhấn mạnh vào sự thay đổi trạng thái.

Nguyễn Ngọc Điệp

Lập trình hàm là gì?

- Trong lập trình mệnh lệnh, các hàm mệnh lệnh có thể tạo ra hiệu ứng lề, làm thay đổi giá trị của một phép tính trước đó.
 - Vì vậy thiếu tính trong suốt tham chiếu, có nghĩa là cùng một biểu thức ngôn ngữ lại có thể tạo ra nhiều giá trị khác nhau vào các thời điểm khác nhau tùy thuộc vào trạng thái của chương trình đang thực thi.
- Trong lập trình hàm, giá trị xuất ra của một hàm chỉ phụ thuộc vào các tham số đầu vào của hàm, vì thế gọi hàm f hai lần với cùng giá trị tham số x sẽ cho ra cùng kết quả $f(x)$.
 - Việc loại bỏ hiệu ứng lề có thể làm cho chương trình dễ hiểu hơn, có thể dự đoán được hành vi của một chương trình.

Nguyễn Ngọc Điệp

Ví dụ:

Tính tổng từ 1 đến 10 trong Java:

```
int total = 0;
for (int i = 1; i ≤ 10; i++)
    total = total + i;
```

Tính toán dựa trên gán các biến → giá trị thay đổi.

15

Ví dụ:

Tính tổng một mảng số nguyên trong Scala:

```
val xs = Array(1, 2, 3, 4, 5, 6, 7, 8,
val s = xs.sum
println(s)
```

Cách tính là ứng dụng hàm.

Cách tư duy của lập trình hàm trong bài toán này là coi x là một mảng, f là hàm tính tổng, và y là kết quả, $y=f(x)$.

16

Ví dụ:

Tính tổng bình phương các phần tử trong mảng số nguyên:

$$s = 1^2 + 3^2 + 5^2 + 7^2 + 9^2$$

```
val r = xs.map(x => x*x).sum
println(r)
```

- Về mặt toán học, ở đây ta sẽ cần sử dụng hai hàm, một hàm tính bình phương của một số và một hàm tính tổng của nhiều số. Trước hết, ta sử dụng hàm map (ánh xạ) tác động lên mảng số để biến mỗi phần tử của mảng đó thành một số chính phương, sau đó mới dùng hàm sum để tính tổng.
- Trong toán, ta gọi đây là phép tổ hợp hàm, hay dùng hàm hợp: $y=g(f(x))$, trong đó f và g là các hàm. Ký hiệu mũi tên $\Rightarrow$ chính là phép biến đổi hàm, chuyển mỗi số vào x thành số ra tương ứng là x^2 .

17

Ưu điểm của Lập trình hàm

- Dễ hiểu hơn, dễ triển khai, kiểm thử và gỡ lỗi.
- Ngắn gọn và súc tích hơn.
- Có thể ít bị lỗi hơn.
- Dễ dàng ứng dụng cho thực thi song song.

MapReduce

Nguyễn Ngọc Điệp

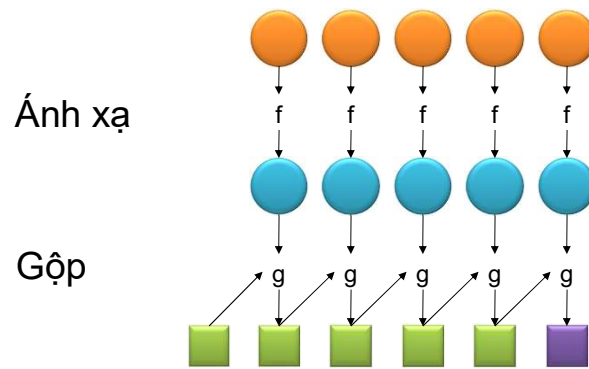
Vấn đề điển hình của dữ liệu lớn

- Duyệt qua một số lượng lớn các bản ghi
- Trích xuất thông tin cần từ mỗi bản ghi trên
- Xáo trộn và sắp xếp kết quả trung gian
- Tổng hợp kết quả trung gian
- Sinh kết quả đầu ra cuối cùng

Ý tưởng cốt lõi: cung cấp một trừu tượng hóa chức năng cho hai hoạt động này

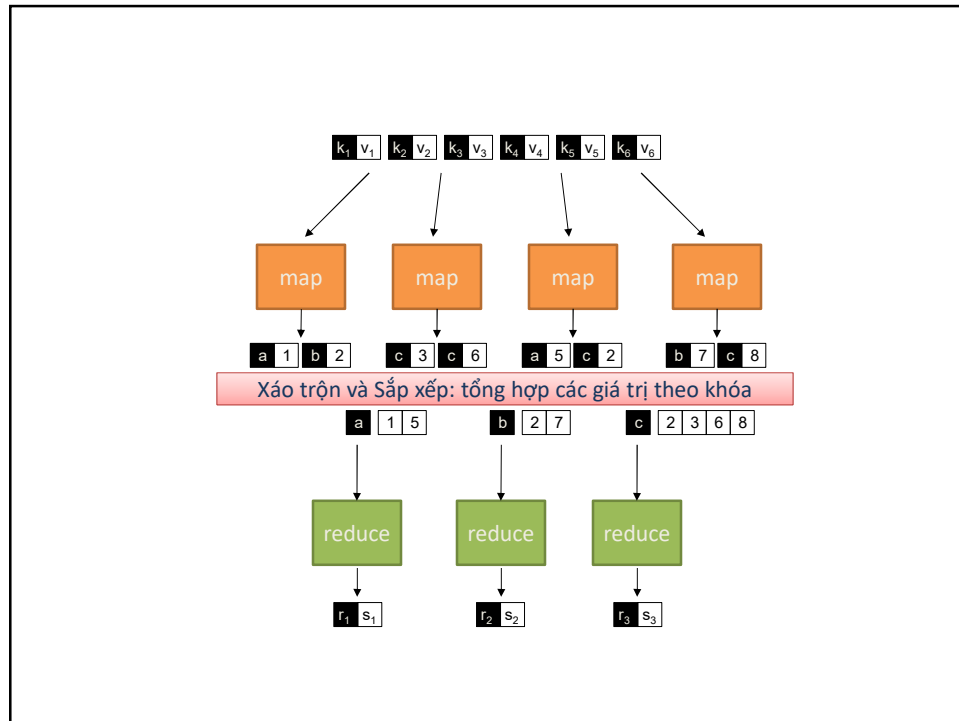
Nguyễn Ngọc Điệp

MapReduce ~ Ánh xạ + Gộp lại nhờ lập trình hàm!



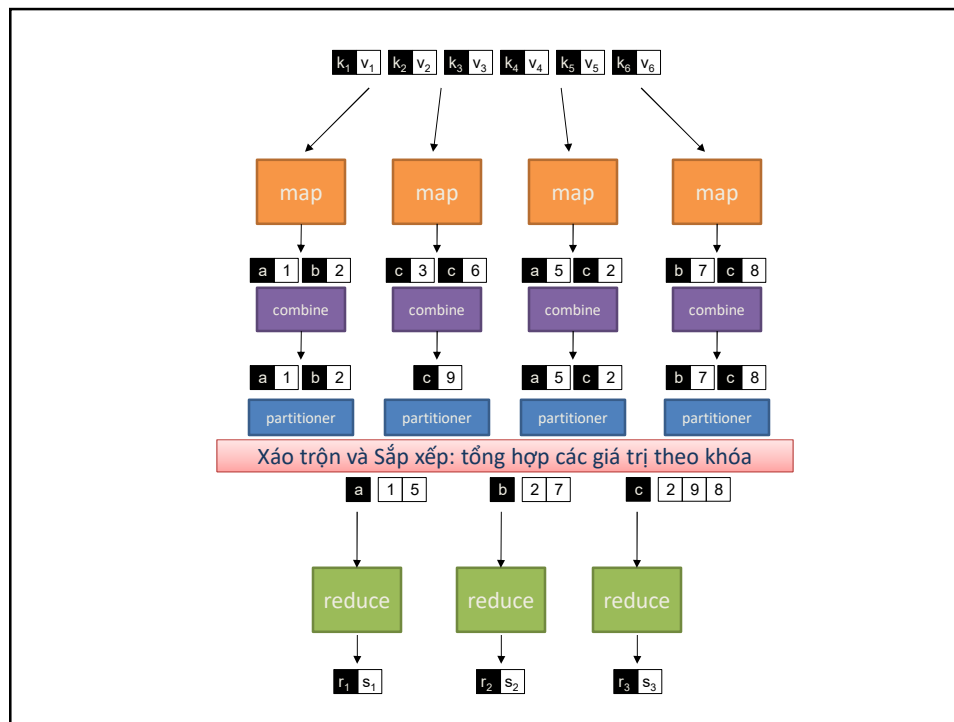
MapReduce

- Lập trình viên xác định 2 hàm:
 - map** $(k, v) \rightarrow \langle k', v' \rangle^*$
 - reduce** $(k', v') \rightarrow \langle k', v' \rangle^*$
 - Tất cả các giá trị có cùng khóa được rút gọn cùng nhau
- Hệ thống xử lý mọi thứ khác...



MapReduce

- Lập trình viên xác định 2 hàm:
 - map** $(k, v) \rightarrow \langle k', v' \rangle^*$
 - reduce** $(k', v') \rightarrow \langle k', v' \rangle^*$
 - Tất cả các giá trị có cùng khóa được rút gọn cùng nhau
- Hệ thống xử lý mọi thứ khác...
- Cũng có lúc, lập trình viên cần xác định:
 - partition** $(k', \text{số partition}) \rightarrow \text{partition cho } k'$
 - Thường là 1 hash đơn giản cho khóa, Ví dụ: $\text{hash}(k') \bmod n$
 - Chia không gian khóa để thực hiện việc rút gọn song song
 - combine** $(k', v') \rightarrow \langle k', v' \rangle^*$
 - Các bộ rút gọn mini (reducer) chạy trong bộ nhớ sau pha ánh xạ (map)
 - Được sử dụng để tối ưu hóa nhằm giảm lưu lượng mạng



Thực thi MapReduce

- Xử lý việc lập kế hoạch
 - Phân công các worker cho nhiệm vụ map và reduce
- Xử lý phân phối dữ liệu
 - Chuyển các tiến trình tới dữ liệu
- Xử lý đồng bộ hóa
 - Tập hợp, sắp xếp và xáo trộn dữ liệu trung gian
- Xử lý lỗi
 - Phát hiện các worker lỗi và khởi động lại
- Mọi thứ xảy ra trên một hệ thống file (FS) phân tán

“Hello World”: Word Count

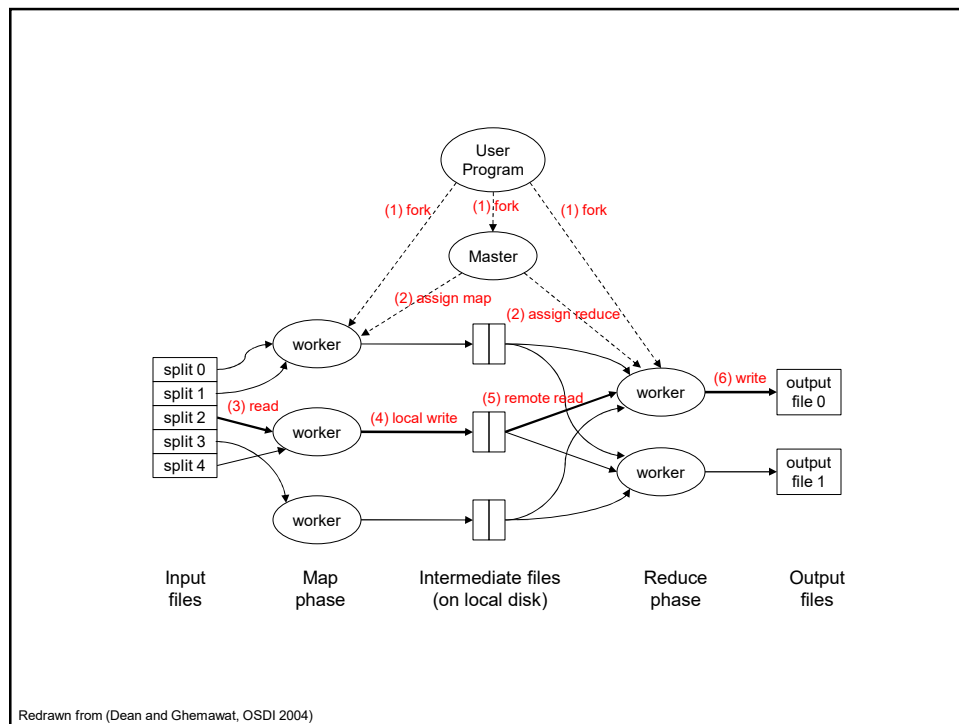
```
Map(String docid, String text):  
  for each word w in text:  
    Emit(w, 1);  
  
Reduce(String term, Iterator<Int> values):  
  int sum = 0;  
  for each v in values:  
    sum += v;  
  Emit(term, value);
```

Nguyễn Ngọc Diệp

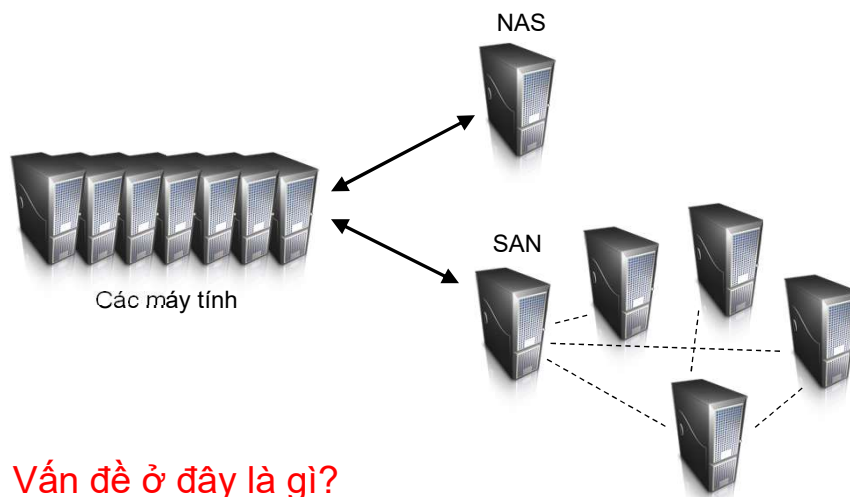
Cài đặt MapReduce

- MapReduce là một mô hình lập trình
- Google có một cài đặt độc quyền trong C ++
 - Các binding cho ngôn ngữ Java, Python
- Hadoop là một cài đặt nguồn mở trong Java
 - Dự án do Yahoo lãnh đạo, được sử dụng trong sản xuất kinh doanh
 - Mở rộng nhanh chóng hệ sinh thái phần mềm

Nguyễn Ngọc Diệp



Làm thế nào chuyển dữ liệu cho các worker?



Hệ thống file phân tán

- Không chuyển dữ liệu cho worker... chuyển worker tới dữ liệu!
 - Lưu trữ dữ liệu trên đĩa cứng của các node tính toán trong cluster
 - Chạy các worker trên node có dữ liệu cục bộ
- Tại sao?
 - Không đủ RAM để lưu tất cả dữ liệu
 - Truy cập đĩa cứng chậm nhưng thông lượng đĩa cứng là có thể.
- Một hệ thống file phân tán sẽ phù hợp:
 - GFS (Google File System)
 - HDFS cho Hadoop (= GFS clone)

Nguyễn Ngọc Điệp

GFS: Các giả thiết

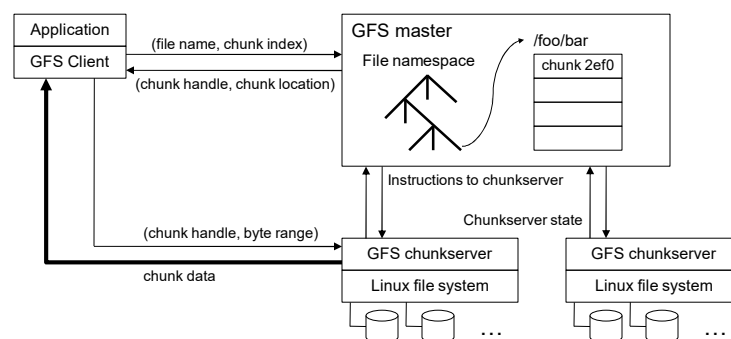
- Sử dụng phần cứng thông thường hơn là các phần cứng chuyên dụng
 - Mở rộng quy mô (scale-out) chứ không nâng cấp hệ thống (scale-up)
 - Tỷ lệ các thành phần lỗi cao
 - Các thành phần phần cứng thông thường thường xuyên xảy ra lỗi
- Số lượng các file lớn ít
- Các file theo kiểu ghi một lần, phần lớn được thêm vào (append)
 - Có thể song song
- Đọc dữ liệu nhiều qua luồng hơn là truy cập ngẫu nhiên
- Thông lượng cao được duy trì hơn là độ trễ thấp

Nguyễn Ngọc Điệp

GFS: Các thiết kế

- File lưu dưới dạng các đoạn (chunk)
 - Kích thước cố định (64MB)
- Tính tin cậy đạt được thông qua nhân bản dữ liệu
 - Mỗi chunk nhân bản hơn 3 lần trên các chunkservers
- Một máy master riêng lẻ để đồng bộ truy cập, lưu trữ metadata
 - Quản trị tập trung đơn giản
- Không đệm dữ liệu
 - Ít lợi ích do bộ dữ liệu lớn, đọc theo luồng
- API đơn giản
 - Đẩy một số vấn đề lên máy khách

Nguyễn Ngọc Điệp



Redrawn from (Ghemawat et al., SOSP 2003)

Trách nhiệm của Master

- Lưu trữ metadata
- Quản lý namespace /khóa
- Liên lạc định kỳ với các chunkserver
- Tạo chunk, tái nhân bản, tái cân bằng
- Thu gom rác (Garbage collection)

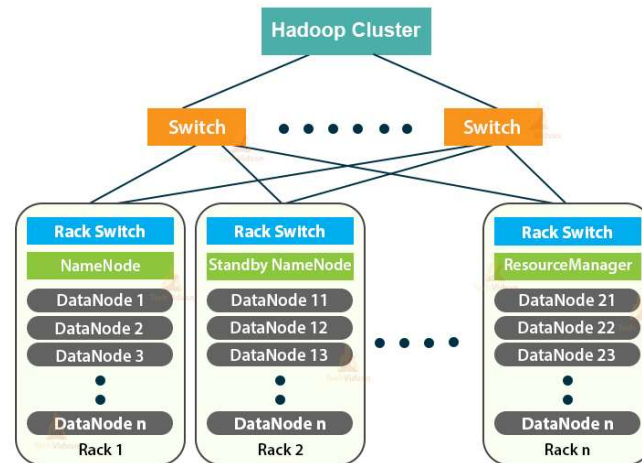
Nguyễn Ngọc Diệp

Kiến trúc phân cụm Hadoop

Nguyễn Ngọc Diệp

Cụm Hadoop

- Tuân thủ theo kiến trúc master-slave



Nguyễn Ngọc Diệp

Cụm Hadoop

- Master trong cụm Hadoop
 - Là một máy tính mạnh với cấu hình bộ nhớ và CPU cao.
 - Hai tiến trình nền gồm NameNode và ResourceManager chạy trên master.

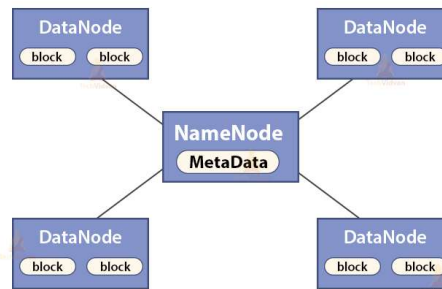
Nguyễn Ngọc Diệp

Cụm Hadoop

- NameNode là nút master trong HDFS Hadoop
 - Quản lý không gian tên hệ thống file. Nó lưu trữ siêu dữ liệu hệ thống tập tin trong bộ nhớ để phục hồi nhanh.

- Các chức năng của NameNode là:

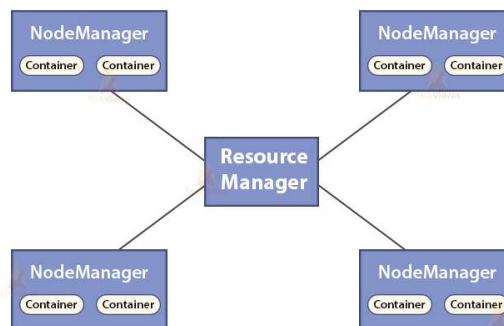
- Quản lý không gian tên hệ thống tập tin
- Lưu trữ dữ liệu meta về các khối dữ liệu của file, vị trí của các khối, quyền, v.v.
- Thực thi các hoạt động không gian tên hệ thống tập tin như mở, đóng, đổi tên tệp và thư mục, v.v.
- Duy trì và quản lý DataNode.



Nguyễn Ngọc Diệp

Cụm Hadoop

- ResourceManager là tiến trình master của YARN
 - ResourceManager phân phối các tài nguyên trong số tất cả các ứng dụng trong hệ thống
 - Theo dõi các nút sống và chết trong cụm



Nguyễn Ngọc Diệp

Cụm Hadoop

- Slave trong cụm Hadoop
 - Phần cứng rẻ tiền.
 - Hai tiến trình nền là DataNodes và YARN NodeManager chạy trên các nút slave.
- Chức năng của DataNodes
 - DataNodes lưu trữ dữ liệu thực tế: các khối của một tập tin
 - Tạo khối, xóa, sao chép dựa trên các hướng dẫn từ NameNode.
 - DataNode chịu trách nhiệm phục vụ các hoạt động đọc / ghi của client.

Nguyễn Ngọc Điệp

Cụm Hadoop

- Chức năng của NodeManager
 - NodeManager là tiến trình nền slave của YARN.
 - Chịu trách nhiệm cho các container, giám sát việc sử dụng tài nguyên của chúng (như CPU, đĩa, bộ nhớ, mạng) và báo cáo với ResourceManager.
 - NodeManager cũng kiểm tra sức khỏe của nút mà nó đang chạy.

Nguyễn Ngọc Điệp

Cụm Hadoop

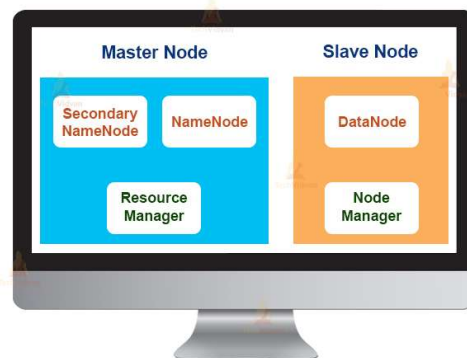
- Nút client trong cụm Hadoop
 - Không phải là nút master cũng không phải là nút slave. Chúng được cài đặt Hadoop với tất cả các cấu hình của cụm.
- Chức năng của các nút Client
 - Tải dữ liệu vào cụm Hadoop.
 - Đề trình các công việc MapReduce, mô tả cách xử lý dữ liệu đó.
 - Lấy kết quả của công việc sau khi xử lý hoàn thành.
- Mở rộng cụm Hadoop:
 - Bằng cách thêm nhiều nút. Hadoop có khả năng mở rộng tuyến tính.
 - Với mỗi nút bổ sung, chúng ta sẽ có được sự gia tăng thông lượng tương ứng, mang lại $(1/n)$ sức mạnh tính toán.

Nguyễn Ngọc Diệp

Cụm Hadoop

- Cụm 1 nút: Tất cả chạy trên 1 nút

Single Node Hadoop Cluster



Nguyễn Ngọc Diệp

Cụm Hadoop

- **Cụm nhiều nút:**
 - Được triển khai trên nhiều máy. Tất cả các tiến trình nền trong cụm Hadoop nhiều nút đều hoạt động và chạy trên các máy khác nhau.
 - Một cụm Hadoop nhiều nút theo kiến trúc master-slave. Tiến trình nền Namenode và ResourceManager chạy trên các nút master là các máy tính cao cấp. Tiến trình nền DataNode và NodeManager chạy trên các nút slave (nút worker), là phần cứng rẻ tiền.
 - Trong cụm Hadoop nhiều nút, các máy slave có thể có mặt ở bất kỳ vị trí nào bất kể vị trí của vị trí vật lý của máy chủ master.

Nguyễn Ngọc Điệp

CÂU HỎI ???

Nguyễn Ngọc Điệp